



STUDENT

0002-DYA

TENTAMEN

250118 DV2606 2110

Salstentamen (DI)

|                 |                  |
|-----------------|------------------|
| Kurskod         | --               |
| Bedömningsform  | --               |
| Starttid        | 18.01.2025 11:00 |
| Sluttid         | 18.01.2025 16:00 |
| Bedömningsfrist | --               |
| PDF skapad      | 05.11.2025 17:13 |
| Skapad av       | Anders Göransson |

- 1.1 You are parallelizing a sequential program. Assume that you manage to parallelize 95% of the sequential program, i.e., 5% is still executed sequentially. What is the highest speedup that you can archive (given an infinite number of processors)?

Select one alternative:

☒ 20



☐ 5

☐ None of the other alternatives is correct

☐ 95

☐ Infinite

---

Rätt. 1 av 1 poäng.

- 1.2 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

☒ All alternatives decrease the efficiency.



☐ Increase the clock frequency of the processors.

☐ Decrease the input data set size.

☐ Increase the number of processors.

☐ Put more barrier synchronizations in the code.

---

Rätt. 1 av 1 poäng.

**1.3** *SPMD* is a common programming model for writing parallel programs. *SPMD* stands for:

- ☐ None of the alternatives is correct.
- ☒ Single Program Multiple Data
- ☐ Shared-memory Program Many Distributions
- ☐ Shared-memory Programming for Multiple Data
- ☐ Single Processor Multiple threads

---

Rätt. 1 av 1 poäng.

**1.4** Simultaneous multithreading is ...

- ☒ a technique to exploit parallelism in the instruction execution pipeline
- ☐ none of the alternatives is correct
- ☐ a software approach to parallelize an application
- ☐ a technique to implement multi-level caches
- ☐ a technique to improve branch prediction

---

Rätt. 1 av 1 poäng.

**1.5** Why are snoopy-based cache coherence protocols not used in large-scale shared-memory multiprocessors?

- ☐ They can not guarantee coherence for more than ten caches.
- ☒ They are not scalable to a large number of nodes. 
- ☐ They can only be implemented using a write-update policy.
- ☐ Modern processors are too fast for snoopy-based protocols.
- ☐ None of the alternatives is correct.

---

Rätt. 1 av 1 poäng.

**1.6** MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ The cache replies with the requested block, and the block is marked as Invalid.
- ☐ The processor is always stalled until the bus request is handled.
- ☐ The cache replies with the requested block, and the block stays in state Modified.
- ☐ None of the alternatives is correct. 

- ☒ The request is denied since the block is in state Modified. 

---

Fel. 0 av 1 poäng.

**1.7** A fundamental difference between a directory-based cache coherence protocol and a snoopy-based cache coherence protocol is ...

- ☐ A directory-based protocol keeps exact information in which caches copies of each memory block reside. ✓
- ☐ A directory-based protocol relies on a shared broadcast medium.
- ☒ A directory-based protocol can only be implemented using the write-invalidate policy. ✗
- ☐ All of the alternatives are correct.
- ☐ A directory-based protocol can only be implemented using the write-update policy.

---

Fel. 0 av 1 poäng.

**1.8** What is the (asymptotic) parallel time complexity of *matrix-matrix multiplication* using  $n$  processors, assuming  $n \times n$  matrices?

- ☐  $O(\log n)$
- ☐  $O(n)$
- ☒ None of the alternatives is correct. ✗
- ☐  $O(n^2)$  ✓
- ☐  $O(n \log n)$

---

Fel. 0 av 1 poäng.

**1.9** A PRAM (Parallel Random Access Machine) can be useful for ...

- ☐ Very easy implementation of parallel hardware.
- ☒ Designing and evaluating parallel algorithms. 
- ☐ All of the alternatives are correct.
- ☐ Running applications with very large input data sets.
- ☐ Running message-passing applications with GPU support.

---

Rätt. 1 av 1 poäng.

**2.1** Describe *Flynn's taxonomy* of different computer architectures.

Flynn's taxonomy lists and explains the different types of computer architectures in relation to the computers capacity to handling instructions and data. These four types of architectures have their different use cases:

- SISD (Single Instruction Single Data): Computation is only done based on one instruction at a given time, as well as on a single data input. Often used in single-core systems due to their limitations in running multiple processes in parallel.
- MISD (Multiple Instruction Single Data): Computation is done based on multiple different instructions, but only on a singular data input. Seldom used, but can be effective in error-handling systems.
- SIMD (Single Instruction Multiple Data): Computation is only done based on one instruction at a given time, but can be done on different parts of the data simultaneously. Most commonly used in GPU:s, thanks to their architecture being focused on having many different processing cores but only being able to run a single kernel function at a given time.
- MIMD (Multiple Instruction Multiple Data): Computation can be done on multiple different instructions on multiple different data inputs. Most common in multi-core CPU:s, where the computer is able to run many different applications simultaneously.

Ord: 194

---

Besvarad.

- 2.2** Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

The main difference between UMA and NUMA multiprocessors lies in the name. UMA has each processor placed at an equal distance to the main memory, while NUMA instead distributes the main memory such that there are different lengths to the memory for each processor. Different length means different latencies, since the time to access a memory block will depend on the speed of light.

While UMA is the more common variant, NUMA has its use cases. For example, in a system where each processor works independently from the others on different parts of the memory, it may decrease the overall latency, making each application run faster.

Ord: 106

---

Besvarad.

**2.3** When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

- When going for the task parallelism approach, you try to split up the application into different and substantial tasks, or functions. These should preferably be independent of each other, meaning that they can run in parallel and aggregate or reduce their results when they all are finished. These tasks are often coarse grained, since having many very small tasks will most likely lead to less efficiency thanks to more idling between processors.

This approach has the advantage of being less prone to data races, as long as these tasks are working on separate memory segments. It is also preferred when needing to run different applications in parallel, such that the user does not have to sit and wait for a process to finish and can instead work on other tasks. The disadvantage of this approach is that it is much harder to make use of many processors, especially for smaller programs, leading to the wall clock time being increased.

Task parallelism is a good choice when making a server-based system, since it should be able to easily serve different clients without them all having to wait for each other. Each request from clients can be seen as a different task and be parallelized as long as there aren't any high data dependencies between the clients. Another suitable application could be a live-streaming platform, where tasks could be divided into the video player, the video stream, the graphical interface, the audience chat, and more. All of these components need to work independently from each other.

- When going for the data parallelism approach, you instead try to divide the work based on the data input. You need to partition the data in a way such that the different processors can work on their own, having all the same instruction. These tasks are often fine grained to maximize performance, as long as there are enough processors available.

The advantage of this approach is that it is easily scalable to the data size. You can simply initialize more threads as the input size increases. It can however backfire if there is many dependencies between the different data elements such that the need for synchronization is high, in which case it will drastically slow down the performance.

Data parallelism is often used for computational tasks with large data structures but with small instructions. These often include image processing, where each pixel has to be modified in a certain way. This application of handling large data sizes is one of the reasons why a GPU is meant to only work on a single instruction at a time, to make use of and encourage data parallelism. It is also a good approach when working on matrix computations, such as matrix multiplication.

Ord: 456

Besvarad.



**2.4** When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on  $N$  processors, we often find that the parallel program is not  $N$  times faster. Describe two important reasons for that.

1. One reason for not being  $N$  times faster is that it is very hard to parallelize the whole program fully. There are often parts that will have to be done sequentially, such as when aggregating the result answer or when having to request data from another process. This is what Amdahl's law explains, that the maximum speedup will be limited by the ratio of the program that has to be run sequentially, which is never zero. There is always some part that has to be run sequentially.
2. Another important reason is that parallelizing a program often introduces excess computation. Different threads will sometimes have to do calculations or memory accesses that some other thread already has done, which will introduce a larger number of instructions overall in comparison to the sequential program.

Ord: 132

---

Besvarad.

**2.5** Sometimes we find that a system or application has a *superlinear speedup*. Clearly describe two situations when and why it may occur.

- When parallelizing an application that has to search through a large number of possibilities to find an answer, such as a depth-first tree traversal, the program can possibly achieve superlinear speedup. This is because it might be that the parallel version does less work overall when compared to the sequential version. In the tree traversal, it might be that the solution would be found within the last branch that the sequential version would explore, while the parallel version had a processor that started on that branch and found it before the others even had time to reach their leaf nodes.
- The other situation would be that there happens to be fewer cache-misses in the parallel version. If you have a large data structure that the sequential version cannot fit into its cache, the parallel version may just have fewer cache misses when partitioning the work based on the input data (data parallelism). It might be that the sequential version has to constantly update the memory blocks within the cache, due to it having to replace the earlier parts of the data structure in the cache, and receiving misses when traversing it once again. The parallel version might get around this problem simply because each of them are working on a smaller part of the data, meaning that each memory block is larger relatively to the cache itself, causing fewer cache misses.

Ord: 231

---

Besvarad.

**2.6 Cache coherence** is an important issue to handle in a shared-memory multiprocessor.

Describe, by giving an example, why ensuring cache coherence is important. (1p)

Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*.

Briefly describe each of them, e.g., by drawing a simple picture. (2p)

Describe some advantage and disadvantage with each of the approaches. (1p)

- Cache coherence is important to make sure that each processor in a multi-core system knows the truth about the variables in the program. Since not all processors will share the same cache, it might be that different processors have different values for the same variable within their respective caches.
  - If  $P_1$  sets the variable  $x=5$  at  $t_1$ , it then puts that information on its cache. If then  $P_2$  sets  $x=7$  at time  $t_2 > t_1$ , then it will store the  $x=7$  in its cache. Since they are writing to the same variable, these instructions has to be synchronized. When  $P_1$  tries to read  $x$ , it will find  $x=5$  in its cache, and print 5 in the terminal, even though it should be 7. This is a problem, since it didn't even matter that we made sure to synchronize the writes! If this problem persists, it can create problems that seem like race conditions but are not.
- The write-invalidate protocol fixes this issue by having a the cache tell the other processors that the variable updated,  $x$ , is no longer valid and needs to be accessed from memory. This can either be done with a directory keeping track of all the variables and which are valid and invalid, or with a snoopy based system by sending the information to all processors through a shared broadcast medium.
- The write-update protocol instead takes the more direct approach by instead of tagging the variables actually broadcasts and updates all of the existing copies of the variable within the different caches. This makes sure that every processor has the right information and won't receive a cache miss when trying to access  $x$ .
- Write-invalidate is useful when the updates to variables are very frequent from a few processors. Simply tagging the variable as invalid is a very light weight operation that does not put too much stress on the memory bus. It does however come with the drawback that if many processors update the variable, it will cause many cache misses, leading to worse performance.
- Write-update instead is better used when the variables are often read and used by different processors. The downside however is that if there is many updates to the same variable in rapid succession without many processes needing to read it, it will be wasted computation time, since the earlier updates are simply written over.

Ord: 393

Besvarad.

### 3.1 Thread coarsening

**Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.**

☒ Thread coarsening can reduce the number redundant calculations in a CUDA program 

☐ Thread coarsening increases the parallelism in a CUDA program

☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program 

☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU

☐ Thread coarsening decreases the completion time of a thread

---

Rätt. 2 av 2 poäng.

### 3.2 Reduction sum

Consider the two functions SumReduction1 and SumReduction2 below:

```
__global__ void SumReduction1(float* input) {
    int i = 2*threadIdx.x
    for (int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x % stride == 0) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}

__global__ void SumReduction2(float* input) {
    int i = threadIdx.x
    for (int stride = blockDim.x; stride >= 1; stride /= 2) {
        if (threadIdx.x < stride) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}
```

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ SumReduction1 uses a gather approach and SumReduction2 uses a scatter approach.
- ☐ SumReduction1 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction2.

☒ SumReduction2 has less control divergence than SumReduction1



☒ SumReduction2 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction1.



☐ SumReduction1 has less control divergence than SumReduction2.

---

Rätt. 2 av 2 poäng.

### 3.3

Consider the following small program.

```
for (int m = 0; m < M; m++) {
    input[m].value += 5;
    for (int n = 0; n < N; n++) {
        output[n].value += input[m].value / distance(input[m].position, output[n].position);
    }
}
```

If we transform this program into a parallel CUDA program, we may run into some serious performance problems in the CUDA program.

What kind of performance problems may the CUDA program suffer from? How could we modify the small program above so that it is still functionally equivalent and so that the corresponding CUDA program does not suffer from any serious performance problems?

This simple program uses a scatter approach, meaning that each input value is having an effect on all output values. Scatter is not a good approach when programming in CUDA, since each thread will be responsible for reading one input element and writing to multiple output elements, all of which are shared between all of the threads. To not cause race conditions, we need to make the write an atomic operation, which will cause the program to run sequentially during the writing to output. Since that is the loop that will run the most times, it will drastically affect performance.

The solution to this problem is to convert this implementation to a functionally identical one that uses gather instead of scatter. Then, each thread would read from multiple input elements and write to a single output element. Having multiple reads on the same memory address by different threads won't cause any race conditions, making this implementation far more efficient.

To perform scatter-to-gather, we need to somehow flip the inner and outer for-loops such that the outer loop corresponds to the selection of a output element, meaning that we want **for (int n=0; n < N; n++)** to be the other most loop that we can then replace with thread assignment. In this example, we do this by first performing a loop fission, such that the first for-loop that adds 5 to all the input values runs to completion in its entirety before going to the output elements. We then need to copy the **for (int m=0; m < M; m++)** down to make the inner for-loop run  $n \cdot m$  times. When these for-loops don't have any statements between them, they are interchangeable, so we can make a loop interchange. This achieves the goal we set out to make, since now we can replace the outer most for-loop with thread creation and makes each thread have responsibility to a single output element. Now we can break out the first for loop and the now second one into their own kernel functions and have it all parallelized without the need for synchronization.

Ord: 350

Besvarad.

## 3.4

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ Context switching on a GPU is normally faster than context switching on a CPU. 

☐ The processor cores in a CPU normally execute in SIMD mode.

☐ A CPU normally has more processor cores than a GPU.

☒ A CPU normally has larger caches than a GPU. 

☐ The processor cores on a CPU are normally organized in streaming multiprocessors.

---

Rätt. 2 av 2 poäng.

## 3.5

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 

☐ All the threads in a thread block see the same registers

☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.

☐ The CPU can access variables in GPU shared memory

☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 

---

Rätt. 2 av 2 poäng.

### 3.6 Cuda Streams

**What are Cuda Streams? How can they be used to improve performance? Give a short example.**

Cuda streams provide more efficient utilization between the CPU and GPU during GPU computations. When you call upon cuda functions, you can specify which stream you want these to flow through. This can be compared to how a thread manages a single control flow through a program, while a Cuda stream manages the control of managing the GPU. Having multiple streams means that you can perform CPU instructions while the GPU works on its own instructions, making them work in parallel.

For example, when allocating and copying memory from the host (CPU) to the device (GPU), the GPU is free to do some computation on the memory it has received in the past. When it has finished its kernel function, it does not have to wait for the CPU to issue more memory to it, and can instead work on a new kernel function immediately. This then makes sure to occupy both CPU and GPU during GPU computation.

Ord: 158

Besvarad.

### 3.7 Thread Blocks and Warps

**Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.**

- ☐ Threads from different Thread Blocks can execute in the same Warp.
- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☒ Threads in the same Warp cannot execute different instructions at the same time. ✓
- ☐ A Thread Block cannot be larger than a Warp.
- ☒ A Thread Block can contain many Warps. ✓

Rätt. 2 av 2 poäng.

### 3.8 Privatization

**Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.**

Privatization works by having a thread block making its own output whilst processing the input before writing to the final output. The idea itself is not limited to being at the thread block level, but is often used when there is a need for different threads to write to the same output elements. Instead of having a large number of threads all trying to write to the same output source, which would require many synchronization locks, each thread block can write to their own output element, which is not as congested and would require less waiting for each thread.

Taking the parallel histogram as an example, you would have each thread counting the number of instances of certain data elements on a partition of the data. Each of the bins of the histogram would have to be incremented each time the thread finds a new element, which with a small number of bins compared to threads would create a large need for synchronization. Instead, having each thread block making its own histogram and then simply adding these together to the final output would create much less waiting for locks, increasing performance. Of course, it can also slow down the program if the data size is too small, but for large data inputs it will be more efficient.

Ord: 217

Besvarad.

### 3.9 Tiling

**Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.**

Tiling is typically used when there is a large part of the input data that is shared between the different threads. In parallel matrix multiplication for example, having one thread calculate one output element, each thread needs a total of  $2N$  elements to be able to calculate that one output element. Instead of having each thread needing to access the global memory to find its  $2N$  number of elements, they can instead work together with the neighboring threads that also need the same input elements and put them into the faster shared memory. Shared memory lies on the streaming multiprocessor and is specific for each thread block. The idea with tiling is that all the threads within a block collaborate to put all of the relevant elements that they all need into shared memory before starting computation. If the number of accesses to shared memory is sufficient, the speedup can be very large. The need for `__syncthreads()` will become apparent to one programming tiling, since the tile needs to be fully loaded into shared memory before any computation can be made. `__syncthreads()` will guarantee that all threads within a block has finished before moving on, making it perfect for this use case.

Ord: 202

Besvarad.



## 3.10

**Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.**

High occupancy means that the cores available on a streaming multiprocessors are better utilized, since there is a higher chance of a warp being ready for execution when it becomes free. This hides the inherent latency that exists when threads have to block for I/O, making sure that despite this, the SM:s are always occupied with work.

Since occupancy is the ratio of active warps to the maximum number of supported warps on the SM, the number of maximum warps supported will be an important parameter to keep track of. This is related to the maximum number of threads per core, and the number of cores that each SM supports. Also, as programmers, we need to keep track of how many threads per block there can be. All of these parameters need to be carefully considered to maximize occupancy.

For example, if there are 1024 maximum threads per block, 8 threads per core, 8 cores per SM, 8 SM:s, 32 threads per warp, and 256 warps per SM, we can start to make calculations as to which different configurations would maximize occupancy.

- 256 warps per SM means that we can have  $256 \cdot 32 = 8192$  threads per SM.
- Having 8 blocks of 1024 threads each would maximize occupancy  $\frac{8 \cdot 1024}{8192} = 1$
- Having 32 blocks of 32 threads each would not maximize occupancy  $\frac{32 \cdot 32}{8192} = \frac{2^5 \cdot 2^5}{2^{13}} = 2^{-3} = \frac{1}{8}$
- 

Ord: 220

Besvarad.